

Variant Calling and Dynamic Programming

Michael Schatz

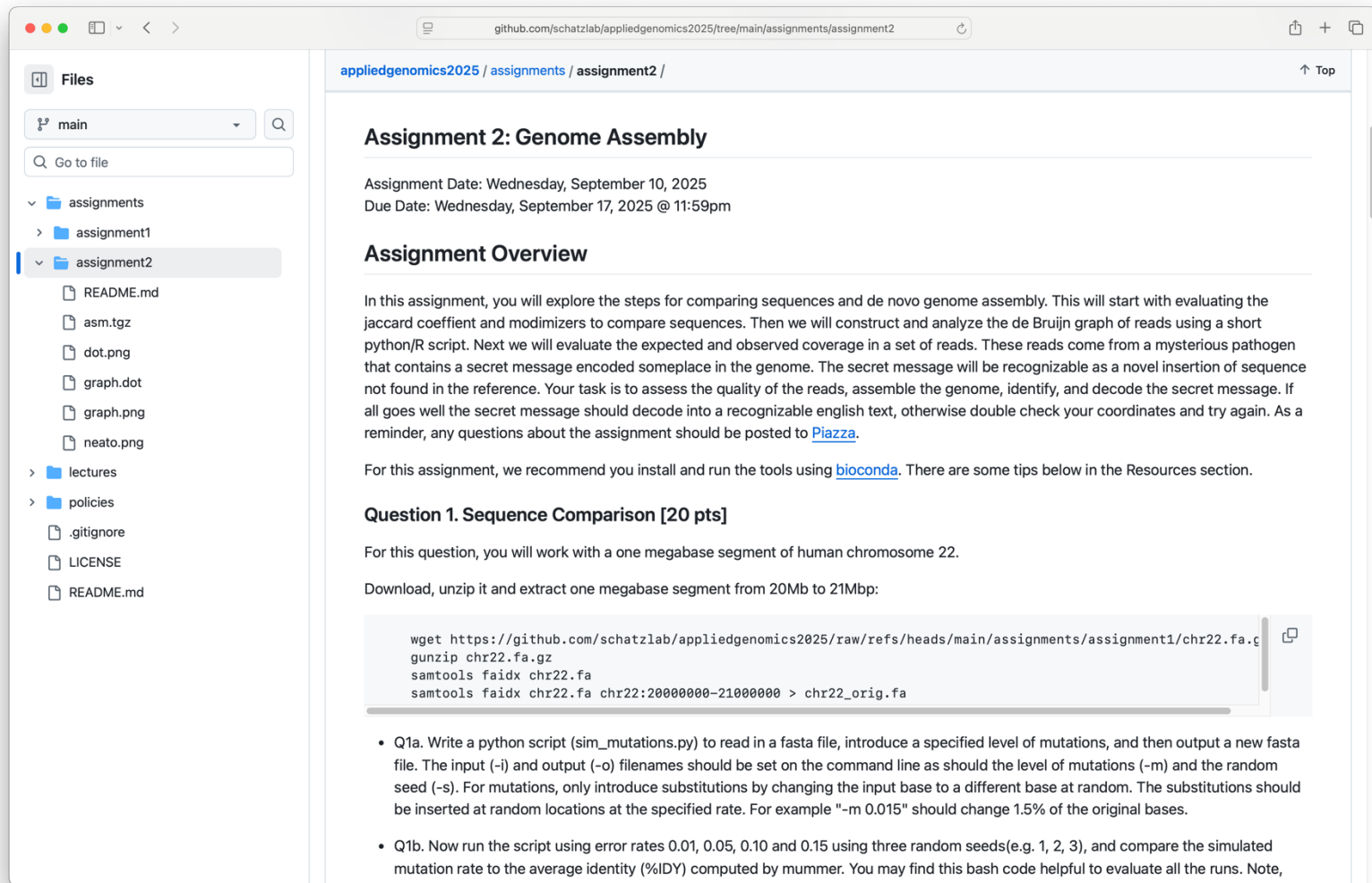
Sept 22, 2025

Lecture 8. Applied Comparative Genomics



Assignment 2: Genome Assembly

Due Wed Sept 17 by 11:59pm



The screenshot shows a web browser displaying the GitHub repository page for 'appliedgenomics2025/assignments/assignment2'. The left sidebar shows the file structure with folders 'assignments', 'assignment1', and 'assignment2'. The 'assignment2' folder is selected, showing files like 'README.md', 'asm.tgz', 'dot.png', 'graph.dot', 'graph.png', 'neato.png', 'lectures', 'policies', '.gitignore', 'LICENSE', and 'README.md'. The main content area displays the 'Assignment 2: Genome Assembly' page. It includes the assignment date (Wednesday, September 10, 2025), the due date (Wednesday, September 17, 2025 @ 11:59pm), and an 'Assignment Overview' section. The overview describes the task: comparing sequences and de novo genome assembly. It mentions evaluating the jaccard coefficient and modimizers, constructing a de Bruijn graph, and analyzing coverage. A secret message is hidden in the genome. The task is to assess the quality of the reads, assemble the genome, identify, and decode the secret message. A reminder to post questions on Piazza is included. Below the overview, there is a section for 'Question 1. Sequence Comparison [20 pts]' which involves working with a one megabase segment of human chromosome 22. It provides instructions to download, unzip, and extract a segment from 20Mb to 21Mb. A code block shows the commands to download the data and extract the segment. Finally, there are two questions: Q1a asks to write a python script to introduce mutations, and Q1b asks to run the script with different error rates and compare the results.

Files

main

Go to file

assignments

assignment1

assignment2

README.md

asm.tgz

dot.png

graph.dot

graph.png

neato.png

lectures

policies

.gitignore

LICENSE

README.md

appliedgenomics2025 / assignments / assignment2 /

Assignment 2: Genome Assembly

Assignment Date: Wednesday, September 10, 2025
Due Date: Wednesday, September 17, 2025 @ 11:59pm

Assignment Overview

In this assignment, you will explore the steps for comparing sequences and de novo genome assembly. This will start with evaluating the jaccard coefficient and modimizers to compare sequences. Then we will construct and analyze the de Bruijn graph of reads using a short python/R script. Next we will evaluate the expected and observed coverage in a set of reads. These reads come from a mysterious pathogen that contains a secret message encoded someplace in the genome. The secret message will be recognizable as a novel insertion of sequence not found in the reference. Your task is to assess the quality of the reads, assemble the genome, identify, and decode the secret message. If all goes well the secret message should decode into a recognizable english text, otherwise double check your coordinates and try again. As a reminder, any questions about the assignment should be posted to [Piazza](#).

For this assignment, we recommend you install and run the tools using [bioconda](#). There are some tips below in the Resources section.

Question 1. Sequence Comparison [20 pts]

For this question, you will work with a one megabase segment of human chromosome 22.

Download, unzip it and extract one megabase segment from 20Mb to 21Mb:

```
wget https://github.com/schatzlab/appliedgenomics2025/raw/refs/heads/main/assignments/assignment1/chr22.fa.gz
gunzip chr22.fa.gz
samtools faidx chr22.fa
samtools faidx chr22.fa chr22:20000000-21000000 > chr22_orig.fa
```

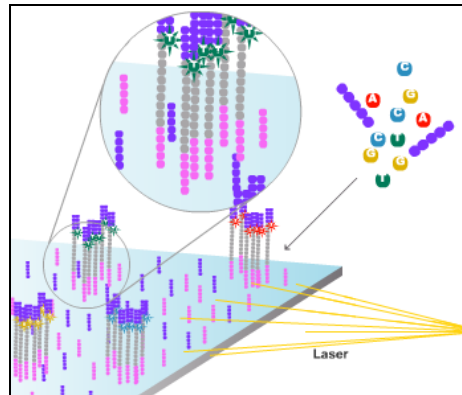
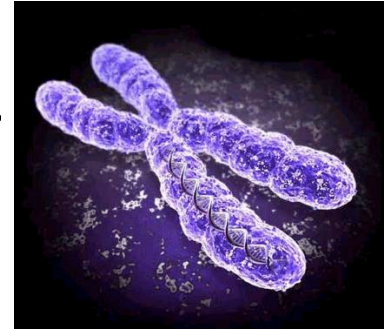
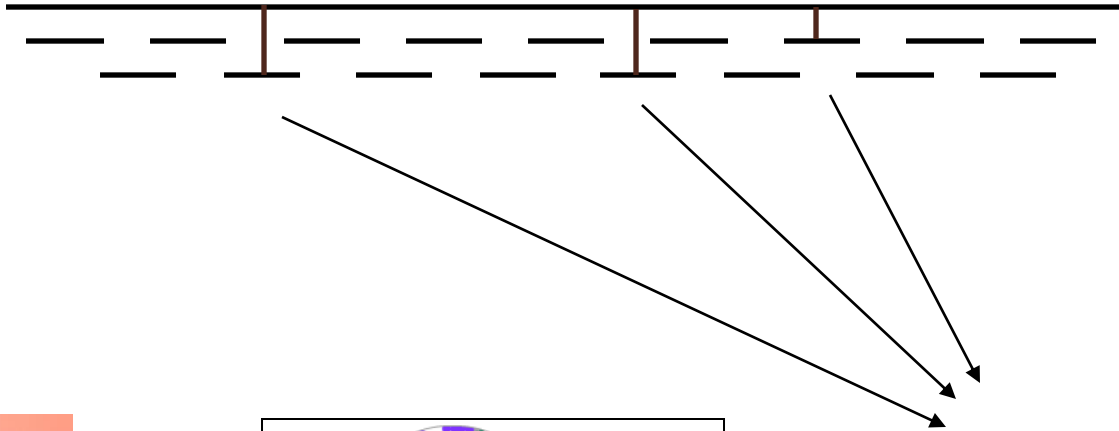
- Q1a. Write a python script (sim_mutations.py) to read in a fasta file, introduce a specified level of mutations, and then output a new fasta file. The input (-i) and output (-o) filenames should be set on the command line as should the level of mutations (-m) and the random seed (-s). For mutations, only introduce substitutions by changing the input base to a different base at random. The substitutions should be inserted at random locations at the specified rate. For example "-m 0.015" should change 1.5% of the original bases.
- Q1b. Now run the script using error rates 0.01, 0.05, 0.10 and 0.15 using three random seeds (e.g. 1, 2, 3), and compare the simulated mutation rate to the average identity (%IDY) computed by mummer. You may find this bash code helpful to evaluate all the runs. Note,

<https://github.com/schatzlab/appliedgenomics2025/tree/main/assignments/assignment2>

Check Piazza for questions!

Personal Genomics

How does your genome compare to the reference?



Heart Disease

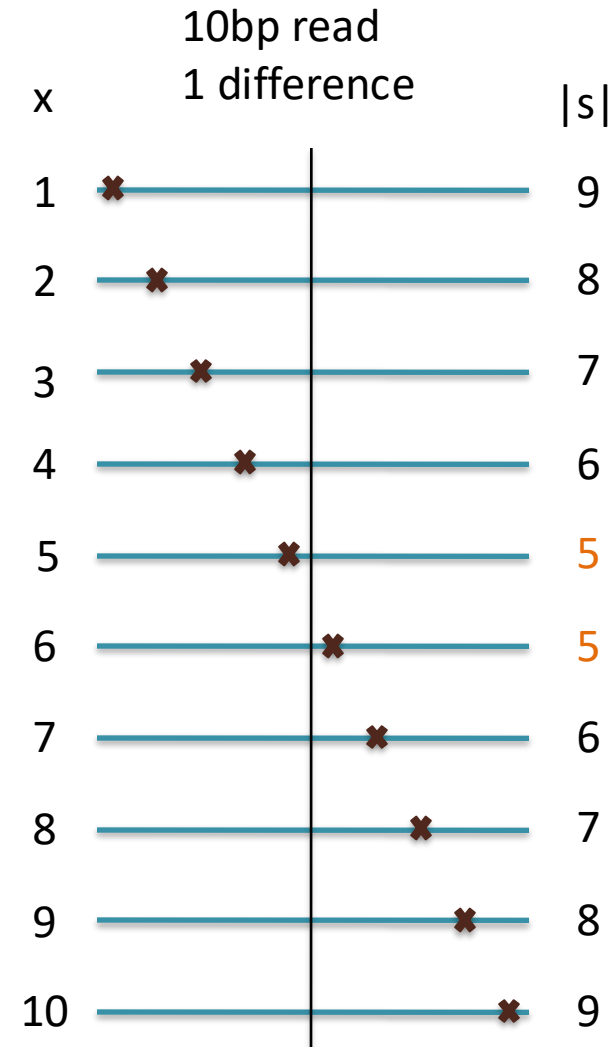
Cancer

Amazing actress

Seed-and-Extend Alignment

Theorem: An alignment of a sequence of length m with at most k differences **must** contain an exact match at least $s = m / (k + 1)$ bp long
(Baeza-Yates and Perleberg, 1996)

- Proof: Pigeonhole principle
 - 1 pigeon can't fill 2 holes
- Seed-and-extend search
 - Use an index to rapidly find short exact alignments to seed longer in-exact alignments
 - BLAST, MUMmer, Bowtie, BWA, SOAP, ...
 - Specificity of the depends on seed length
 - Guaranteed sensitivity for k differences
 - Also finds some (but not all) lower quality alignments <- heuristic



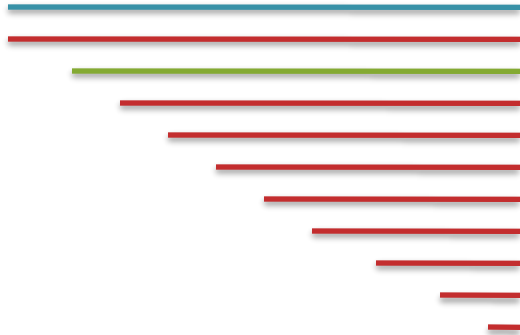
Brute Force Analysis



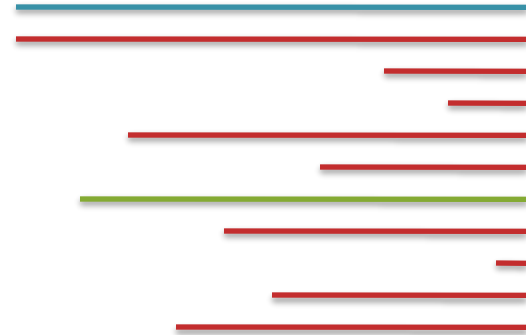
- Brute Force:
 - At every possible offset in the genome:
 - Do all of the characters of the query match?
- Analysis
 - Simple, easy to understand
 - Genome length = n [3B]
 - Query length = m [7]
 - Comparisons: $(n-m+1) * m$ [21B]
- Overall runtime: $O(nm)$
 - [How long would it take if we double the genome size, read length?]
 - [How long would it take if we double both?]

Suffix Arrays: Searching the Phone Book

- What if we need to check many queries?
 - We don't need to check every page of the phone book to find 'Schatz'
 - Sorting alphabetically lets us immediately skip 96% (25/26) of the book *without any loss in accuracy*
- Sorting the genome: Suffix Array (Manber & Myers, 1991)
 - Sort every suffix of the genome



Split into n suffixes



Sort suffixes alphabetically

[Challenge Question: How else could we split the genome?]

Binary Search Analysis

- Binary Search

Initialize search range to entire list

$\text{mid} = (\text{hi} + \text{lo}) / 2$; $\text{middle} = \text{suffix}[\text{mid}]$

if query matches middle: done

else if query < middle: pick low range

else if query > middle: pick hi range

Repeat until done or empty range

[WHEN?]

- Analysis

- More complicated method

- How many times do we repeat?

- How many times can it cut the range in half?

- Find smallest x such that: $n / (2^x) \leq 1$; $x = \lg_2(n)$

[32]

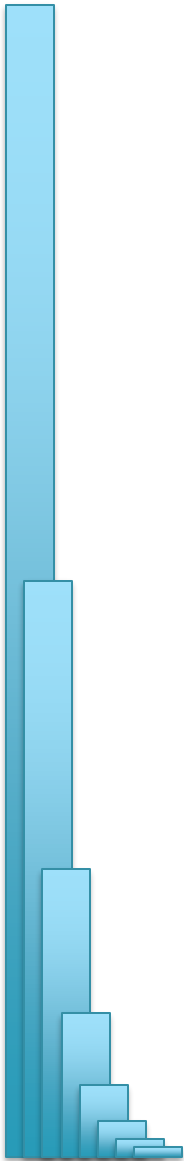
- Total Runtime: $O(m \lg n)$

- More complicated, but **much** faster!

- Looking up a query loops 32 times instead of 3B

Can be reduced to $O(m + \lg n)$

using an auxiliary data structure called the LCP array



Algorithmic challenge

How can we combine the speed of a suffix array $O(m + \lg(n))$ (or even $O(m)$) with the size of a brute force analysis (n bytes)?

What would such an index look like?



Bowtie: Ultrafast and memory efficient alignment of short DNA sequences to the human genome

Slides Courtesy of Ben Langmead

Algorithm Overview

1. Split read into segments

Read

CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA

Read (reverse complement)

TACAGGCCTGGGTAAAATAAGGCTGAGAGCTACTGG

Policy: extract 16 nt seed every 10 nt

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: TCAGCCTTATTTACC

+, 20: **TTACCCAGGCCTGTA**

-, 0: **TACAGGCCTGGGTAAA**

-, 10: GGTAAAATAAGGCTGA

-, 20: GGCTGAGAGCTACTGG

2. Lookup each segment and prioritize

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: **TCAGCCTTATTTACC**

+, 20: **TTTACCCAGGCCTGTA**

-, 0: **TACAGGCCTGGGTAAA**

-, 10: GGTAAAATAAGGCTGA

-, 20: GGCTGAGAGCTACTGG

Ungapped
alignment with
FM Index

Diagram illustrating a sequence alignment between a reference sequence and a query sequence. The reference sequence is "a a c" and the query sequence is "\$ a c a a c g". The alignment shows the query sequence being mapped to the reference sequence, with a blue "I" indicating an insertion and red arrows showing the mapping of "c" to "a" and "g" to "a".

Seed alignments (as B ranges)

$$\{ [211, 212], [212, 214] \}$$
$$\{ [653, 654], [651, 653] \}$$

{ [684, 685] }

{ }

{ }

{ [624, 625] }

3. Evaluate end-to-end match

Extension candidates

SA:684, chr12:1955

SA:624, chr2:462

SA:211: chr4:762

SA:213: chr12:1935

SA:652: chr12:1945

SIMD dynamic
programming
aligner

SAM alignments

```

r1      0      chr12      1936      0
      36M      *      0      0
      CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA
      IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
      AS:i:0      XS:i:-2      XN:i:0
      XM:i:0      XO:i:0      XG:i:0
      NM:i:0      MD:Z:36      YT:Z:UU
      YM:i:0

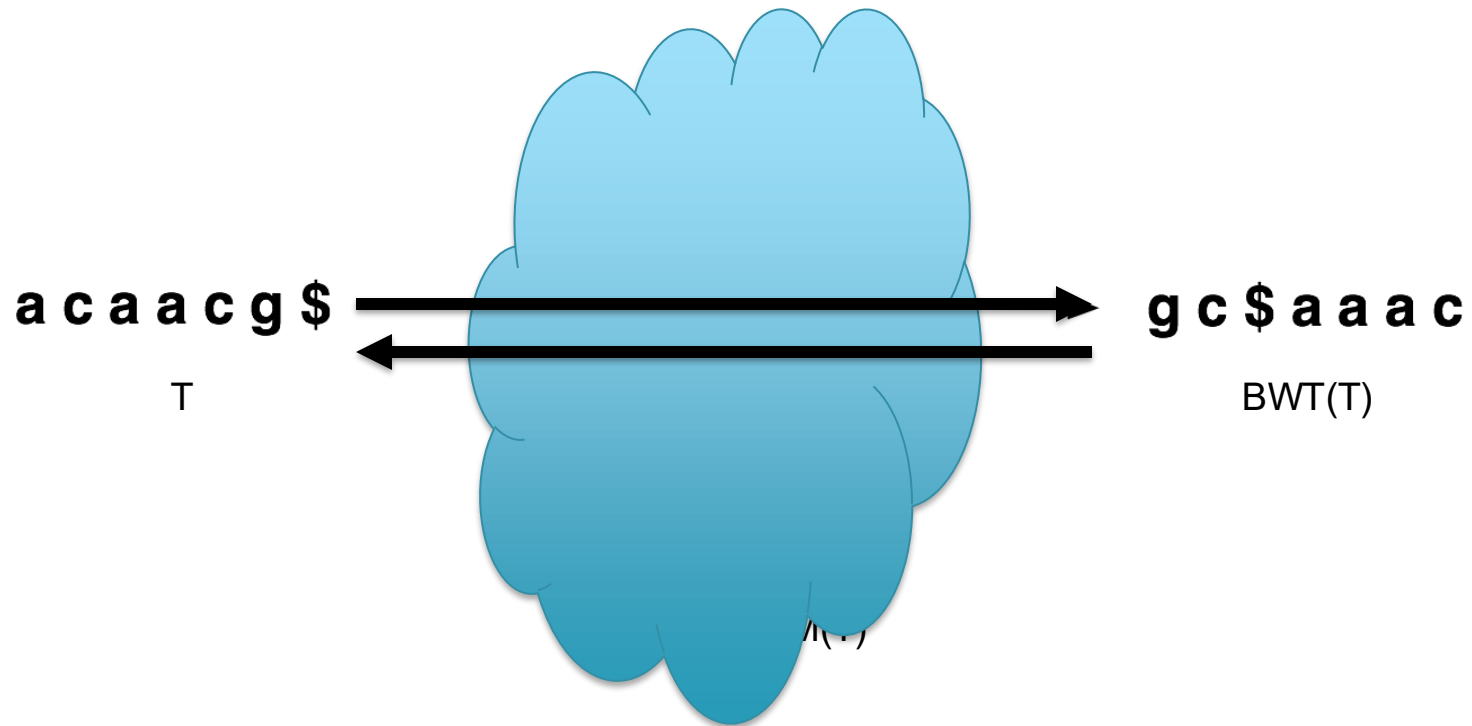
```

□ □

(Langmead & Salzberg, 2012)

Burrows-Wheeler Transform

- Reversible permutation of the characters in a text

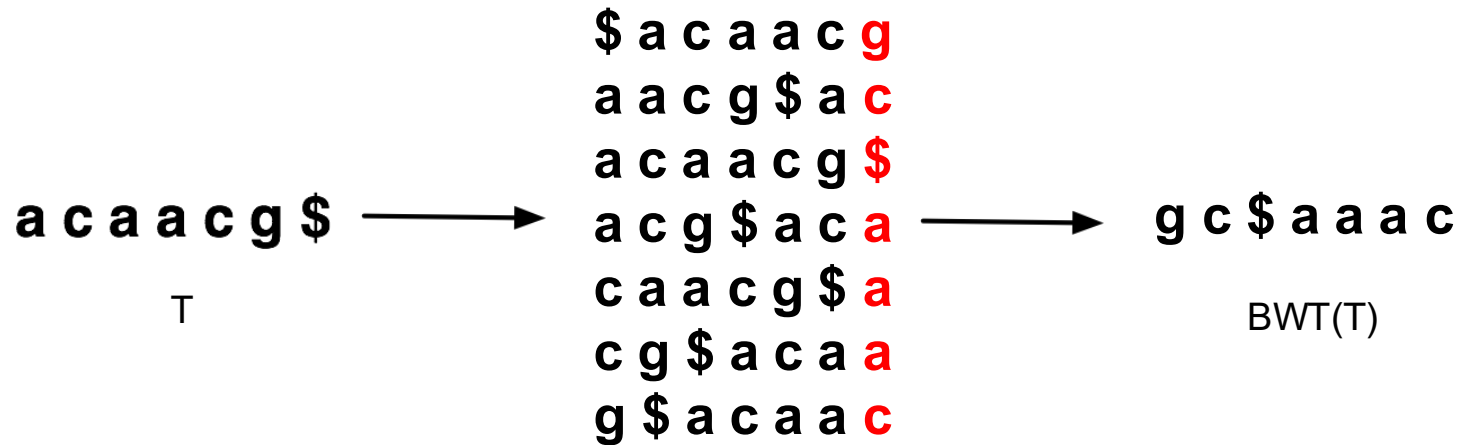


A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text



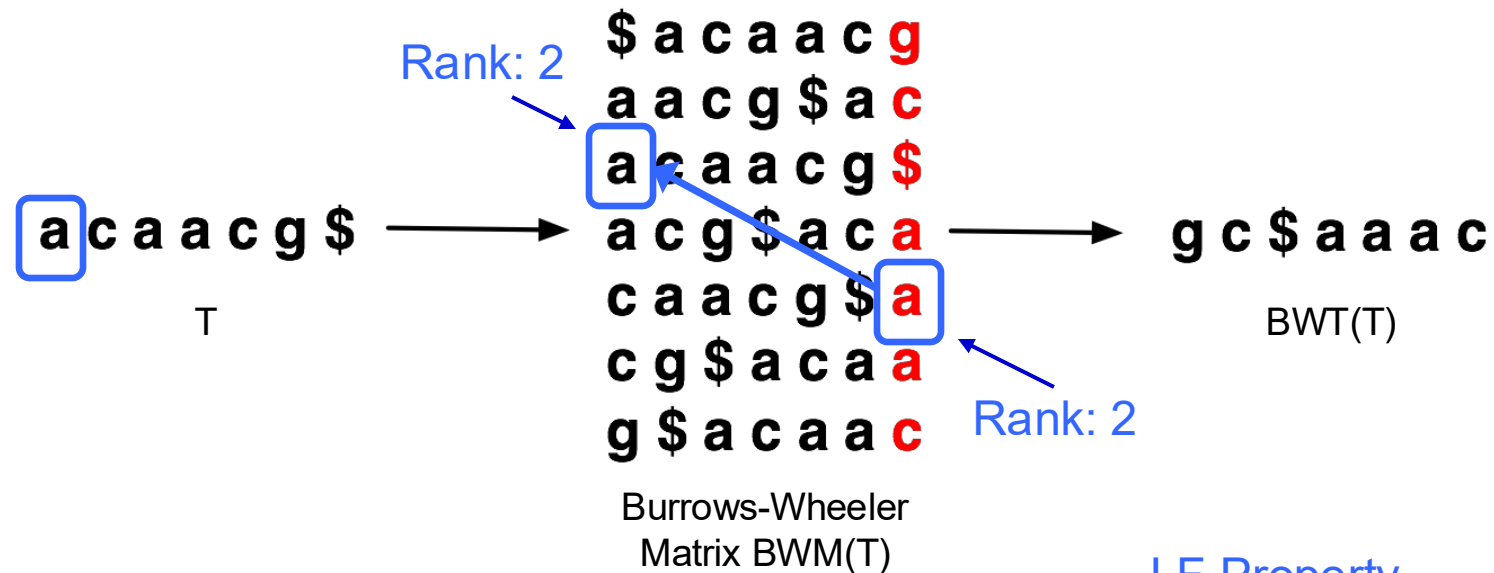
$BWT(T)$ is the index for T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Reversible permutation of the characters in a text

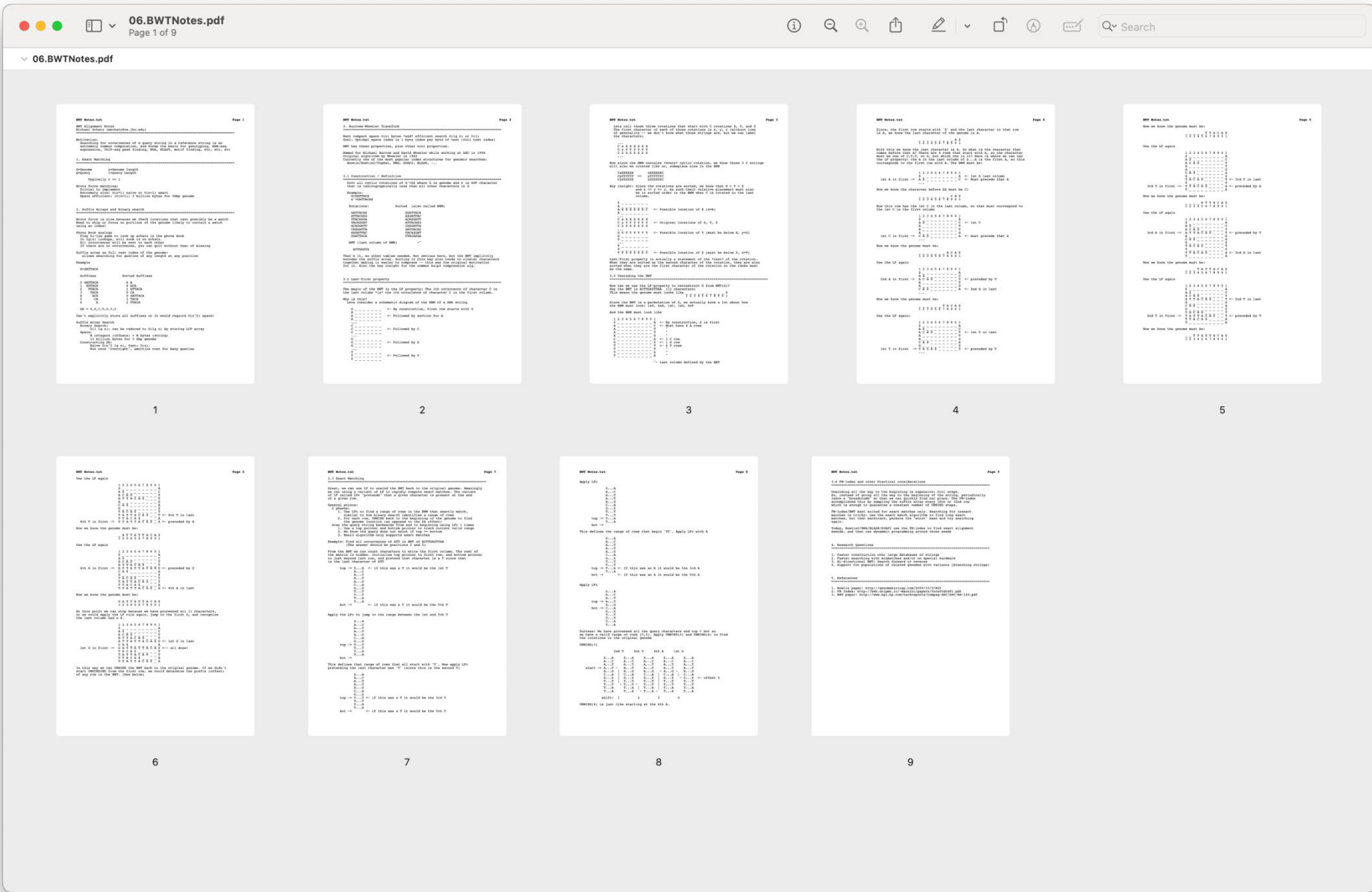


LF Property
implicitly encodes
Suffix Array

- BWT(T) is the index for T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124



Recompute the original text when the BWT is ACTTGA\$TTAA

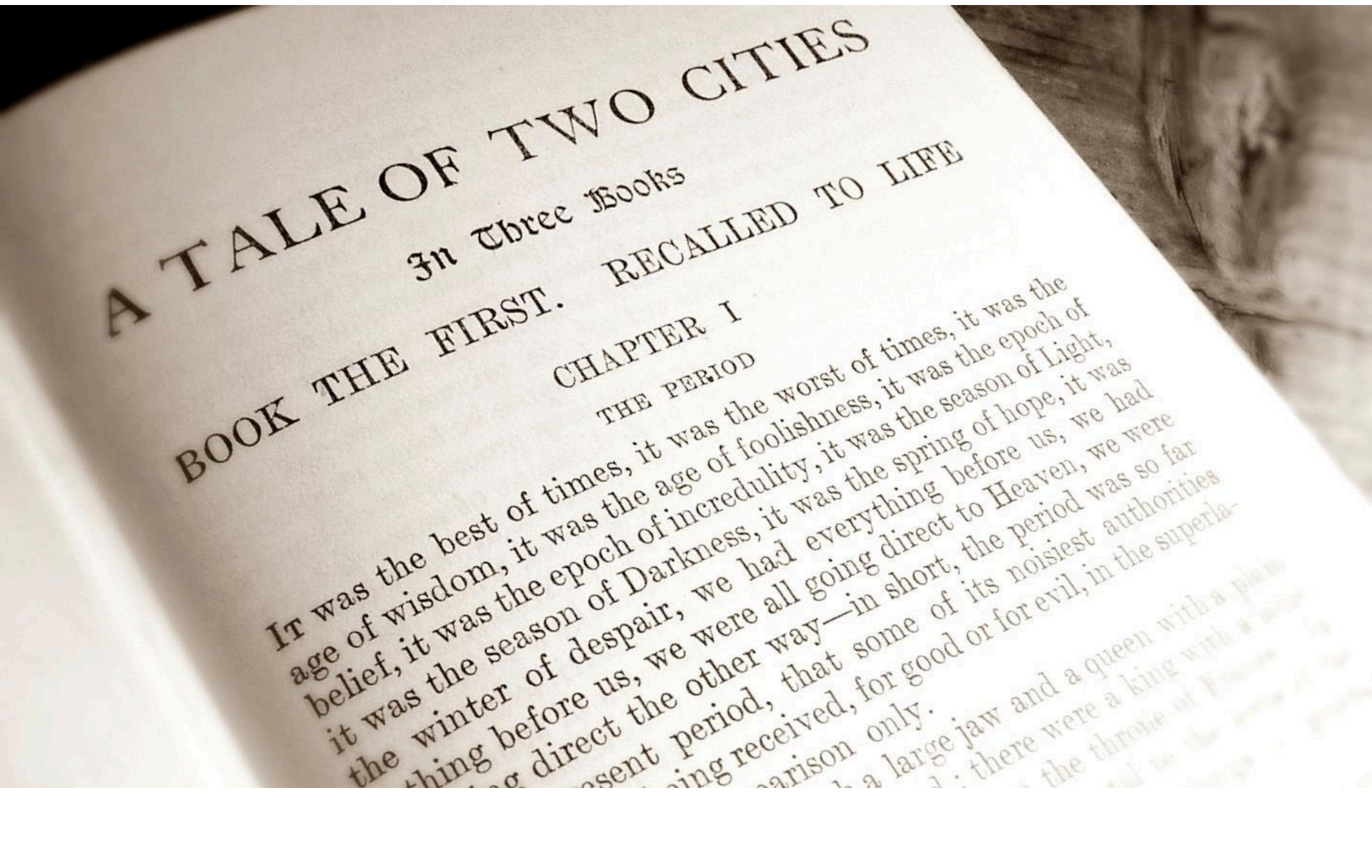
Burrows-Wheeler Transform

- Recreating T from BWT(T)
 - Start in the first row and apply **LF** repeatedly, accumulating predecessors along the way



[Decode this BWT string: ACTGA\$TTA]

Run Length Encoding



Run Length Encoding

ref[614] :

```
It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_
of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief
,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa
s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint
er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us
,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o
ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_
that_some_of_its_noisiest_authorities_insisted_on_its_being_received
,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.$
```

Run Length Encoding:

- Replace a “run” of a character X with a single X followed by the length of the run
- GAAAAAAAAATTACA => GA8T2ACA (reverse is also easy to implement)
- If your text contains numbers, then you will need to use a (slightly) more sophisticated encoding

Run Length Encoding

ref[614] :

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_ that_some_of_its_noisiest_authorities_insisted_on_its_being_received ,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.\$

rle(ref) [614] :

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_fo2lishnes2,_it_was_the_epoch_of_belief ,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darknes2,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us ,_we_were_al2_going_direct_to_Heaven,_we_were_al2_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_ that_some_of_its_noisiest_authorities_insisted_on_its_being_received ,_for_go2d_or_for_evil,_in_the_superlative_degre2_of_comparison_only.\$

Run Length Encoding

ref[614] :

```
It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_
of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief
,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa
s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint
er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us
,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o
ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_
that_some_of_its_noisiest_authorities_insisted_on_its_being_received
,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.$
```

bwt[614] :

```
.dlmssftysestdtrsns_y__$yfofeeeetggsfefefggeedrofr,llreef-,fs,,,,,,
,,nfrsdnnhereghettedndeteegenstee,sssst,esssnssffteedttttttttttr,,
,,eeefehh__p__fpDwwwwwwwwweehl_ew_____eoo_neeeoaaeoo_____sephhrrhvh
hwwegmghhhhhhhkrrwwhhssHrrrvtrribbdbcbvs__thwwpppvmmirdnnib__eoooooo
oooooo_____eennnnnnnaai__ecc__ttttttttttttttttttts_tsgltsLlvtt__hhoor
e_wrraddwlors_____r__lteirillre_ouaanooiioeooooiihkiiiiio__iei
tsppioi_____ggnodsc_sss_gfhf_fffhwh_nsmo__uee_sioooaeeeeoo__ii
cgppeeaoaeooeesseuutetaaaaaaaaaaaaaai__ei_in__aaie_eeerei_hrsssnacciiIi
iiiiisn_____oyoui__a_iids__aiaae_____tlar
```

Run Length Encoding

ref[614] :

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_ that_some_of_its_noisiest_authorities_insisted_on_its_being_received ,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.\$

bwt[614] :

.dlmssftysestdtrsns_y__\$yfofeeeetggsfefefggeedrofr,llreef-,fs,,,,,,,,, ,nfrsdnnhereghettedndeteegenstee,sssst,esssnssffteedttttttttttr,, ,eeefehh_p__fpDwwwwwwwwweehl_ew_____eoo_neeeoaaeoo_____sephhrrhvh hwwegmghhhhhhhkrrwwhhssHrrrvtrribbdbcbvs__thwwpppvmmirdnnib__eoooooo oooooo_____eennnnnnaai__ecc__ttttttttttttttttttts_tsgltsLlvtt__hhoor e_wrraddwlors_____r__lteirillre_ouaanooiioeooooiihkiiiiio__iei tsppioi_____ggnodsc_sss_gfhf_fffhwh_nsmo__uee_sioooaeeeeoo__ii cgppeeaoaeooeesseuutetaaaaaaaaaaaai__ei_in__aaie_eeerei_hrsssnacciiIi iiiiiisn_____ Why does the BWT tend to make runs in english text? _____tlar

Run Length Encoding

bwt[614] :

```
.dlmssftysesdtrsns_y__$yfofeeeeetggsfefefggeedrofr,llreef-,fs,,,,,,  
,,nfrsdnnherghettedndetegeeenstee,sssst,esssnssffteedttttttttttr,,  
,,eeefehh__p__fpDwwwwwwwwweehl_ew_____eoo_neeeoaaeoo_____sephhrrhvh  
hwwegmghhhhhhhkrrwwhhssHrrrvtrribbdbcbvs__thwwpppvmmirdnnib__eoooooo  
oooooo_____eennnnnnaai__ecc__ttttttttttttttttts_tsgltsLlvt__hhoor  
e_wrraddwlors_____r__lteirillre_ouaanooioeooooiihkiiiiio__iei  
tsppioi_____ggnodsc_sss_gfhf_fffhwh_nsmo__uee_sioooaeeeeoo__ii  
cgppeeaoaeooesseuutetaaaaaaaaaaaaai__ei_in__aaie_eeerei_hrssnacciiIi  
iiiiisn_____oyoui__a_iids__aiaae_____tlar
```

rle(bwt) [464] :

```
.dlms2ftysesdtrsns_y_2$yfofe4tg2sfefefg2e2drofr,l2re2f-,fs,9nfrsdn2  
herghet2edndete2ge2nste2,s5t,es3ns2f2te2dt10r,4e3feh2_2p_2fpDw11e2h  
l_ew_5eo2_ne3oa2eo2_4seph2r2hvh2w2egmgh7kr2w2h2s2Hr3vtr2ib2dbcbvs_2t  
hw2p3vm2irdn2ib_2eo12_4e2n6a2i_3ec2_2t18s_tsgltsLlvt2_3h2o2re_wr2ad2  
wlors_9r_2lteiril2re_oua2no2i2oeo4i3hki6o_2ieitsp2ioi_12g2nodsc_s3_g  
fhf_f3hwh_nsmo_2ue2_sio3ae4o2_i2cgp2e2aoaeo2e2s2eu2teta11i_2ei_in_2a  
2ie_e3rei_hrs3nac2i2Ii7sn_15oyoui_2a_i3ds_2ai2ae2_21tlar
```

Run Length Encoding

bwt[614] :

```
.dlmssftysesdtrsns_y__$_yfofeeeeetggsfefefggeedrofr,llreef-,fs,,,,,,  
,,nfrsdnnherghettedndeteegeenstee,sssst,esssnssffteedttttttttttr,,  
,,eeefehh__p__fpDwwwwwwwwweehl_ew_____eoo_neeeoaaeoo_____sephhrrhvh  
hwwegmghhhhhhhkrrwvhhssHrrrvtrribbdbcbvs__thwppppvmmirdnnib__eoooooo  
oooooo_____eennnnnnaai__ecc__ttttttttttttttts_tsgltsLlvt__hhoor  
e_wrraddwlors_____r__lteirillre_ouaanooiioeooooiihkiiiiio__iei  
tsppioi_____ggnodsc_sss_gfhf_fffhwh_nsmo__uee_sioooaeeeeoo__ii  
cgppeeaoaeooeesseuutetaaaaaaaaaai__ei_in__aaie_eeerei_hrsssnacciiIi  
iiiiisn_____oyoui__a_iids__aiaae_____tlar
```

rle(bwt) [464] :

```
.dlms2ftysesdtrsns_y_2$_yfofe4tg2sfefefg2e2drofr,l2re2f-,fs,9nfrsdn2  
herghet2edndete2ge2nste2,s5t,es3ns2f2te2dt10r,4e3feh2_2p_2fpDw11e2h  
l_ew_5eo2_ne3oa2eo2_4seph2r2hvh2w2egmgh7kr2w2h2s2Hr3vtr2ib2dbcbvs_2t  
hw2p3vm2irdn2ib_2eo12_4e2n6a2i_3ec2_2t18s_tsgltsLlvt2_3h2o2re_wr2ad2  
wlors_9r_2lteiril2re_oua2no2i2oeo4i3hki6o_2ieitsp2ioi_12g2nodsc_s3_g  
fhf_f3hwh_nsmo_2ue2_sio3ae4o2_i2cgp2e2aoaeo2e2s2eu2tet a11i_2ei_in_2a  
2ie_e3rei_hrs3nac2i2Ii7sn_15oyoui_2a_i3ds_2ai2ae2_21tlar
```

Run Length Encoding

ref[614] :

```
It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_
of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief
,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa
s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint
er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us
,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o
ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_
that_some_of_its_noisiest_authorities_insisted_on_its_being_received
,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.$
```

rle(bwt) [464] :

```
.dlms2ftysesdtrsns_y_2$_yfofe4tg2sfefefg2e2drofr,l2re2f-,fs,9nfrsdn2
hereghet2edndete2ge2nste2,s5t,es3ns2f2te2dt10r,4e3feh2_2p_2fpDw11e2h
l_ew_5eo2_ne3oa2eo2_4seph2r2hvh2w2egmgh7kr2w2h2s2Hr3vtr2ib2dbcbvs_2t
hw2p3vm2irdn2ib_2eo12_4e2n6a2i_3ec2_2t18s_tsgltsLlvt2_3h2o2re_wr2ad2
wlors_9r_2lteiril2re_oua2no2i2oeo4i3hki6o_2ieitsp2ioi_12g2nodsc_s3_g
fhf_f3hwh_nsmo_2ue2_sio3ae4o2_i2cgp2e2aoaeo2e2s2eu2tet11i_2ei_in_2a
2ie_e3rei_hrs3nac2i2Ii7sn_15oyoui_2a_i3ds_2ai2ae2_21tlar
```

Run Length Encoding

ref[614] :

```
It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_
of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief
,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa
s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint
er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us
,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o
ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_
that_some_of_its_noisiest_authorities_insisted_on_its_being_received
,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.$
```

rle(bwt) [464] :

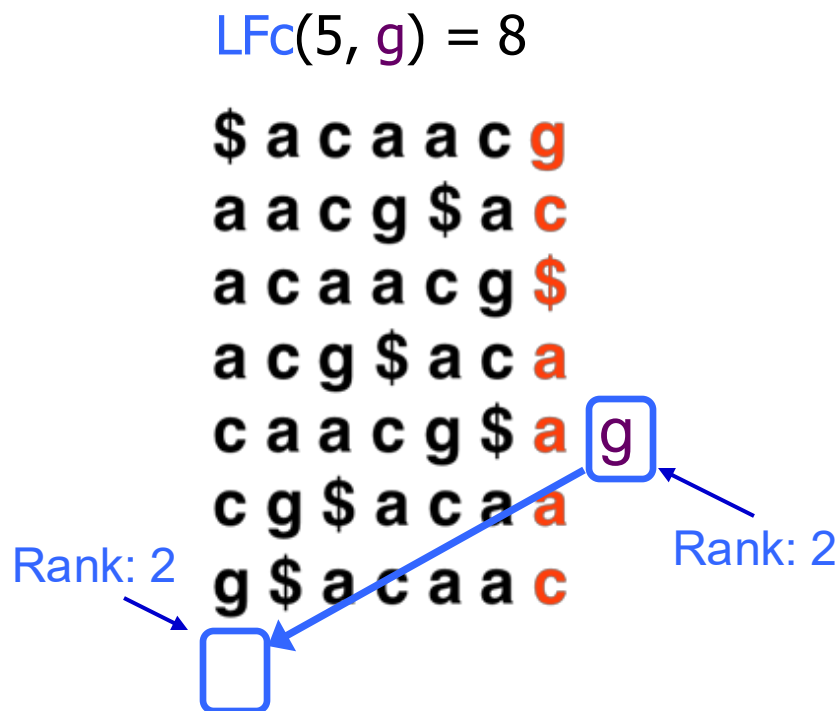
```
.dlms2ftysesdtrsns_y_2$_yfofe4tg2sfefefg2e2drofr,l2re2f-,fs,9nfrsdn2
hereghet2edndete2ge2nste2,s5t,es3ns2f2te2dt10r,4e3feh2_2p_2fpDw11e2h
l_ew_5eo2_ne3oa2eo2_4seph2r2hvh2w2egmgh7kr2w2h2s2Hr3vtr2ib2dbcbvs_2t
hw2p3vm2irdn2ib_2eo12_4e2n6a2i_3ec2_2t18s_tsgltsLlvt2_3h2o2re_wr2ad2
wlors_9r_2lteiril2re_oua2no2i2oeo4i3hki6o_2ieitsp2ioi_12g2nodsc_s3_g
fhf_f3hwh_nsmo_2ue2_sio3ae4o2_i2cgp2e2aoaeo2e2s2eu2tet11i_2ei_in_2a
2ie_e3rei
```

Saved 614-464 = 150 bytes (24%) with zero loss of information!

Common to save 50% to 90% on real world files with bzip2

BWT Exact Matching

- **LFc**(r, c) does the same thing as **LF**(r) but it ignores r's actual final character and “pretends” it's c:

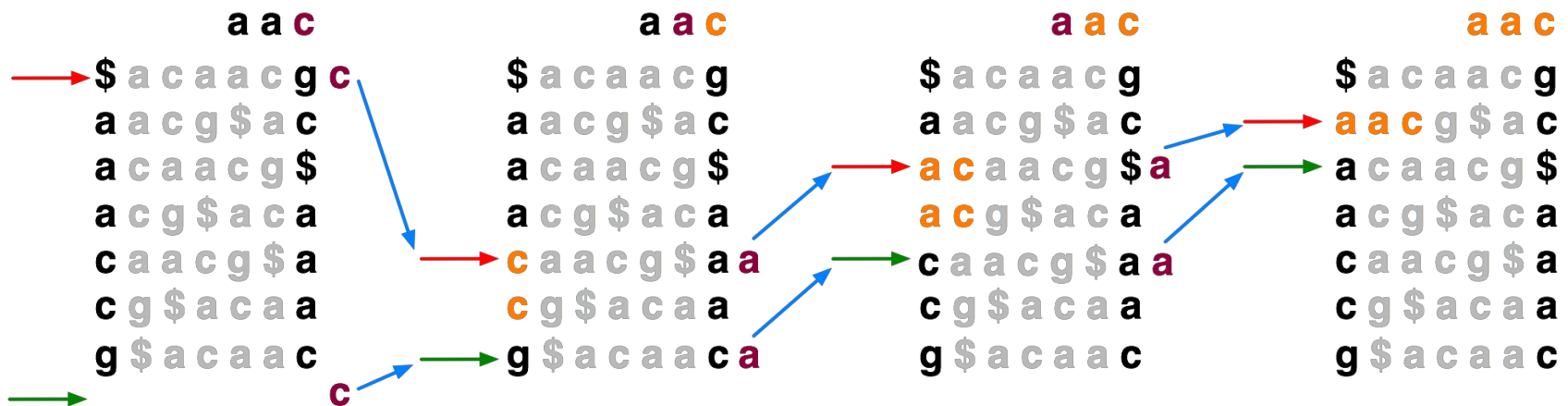


BWT Exact Matching

- Start with a range, (**top**, **bot**) encompassing all rows and repeatedly apply **LFc**:

$$\text{top} = \text{LFc}(\text{top}, \text{qc}); \text{bot} = \text{LFc}(\text{bot}, \text{qc})$$

qc = the next character to the left in the query



Ferragina P, Manzini G: Opportunistic data structures with applications. *FOCS. IEEE Computer Society; 2000.*

[Search for TTA this BWT string: ACTGA\$TTA]

FM Index



\$	a	c	a	a	c	g
a	a	c	g	\$	a	c
a	c	a	a	c	g	\$
a	c	g	\$	a	c	a
c	a	a	c	g	\$	a
c	g	\$	a	c	a	a
g	\$	a	c	a	a	c

Where is this row in the original text?

- Could repeatedly apply LF transformation, but could require many steps: $O(N)$
- Instead, periodically record the position: sample the suffix array every 100th row

Which 'a' is this?

- Could count from the top of the matrix, but this will be very slow: $O(n)$
- Instead, periodically (every 100th row) keep track of the number of occurrences of the characters that have seen so far: \$:1, a:2, c:1, g:1, t:0

FM-index: BWT + SA sample + count table

3Gb + (30M ints) + (30M * 5 ints)

3Gb + 120Mb + 600Mb = 3.72Gb

Opportunistic Data Structures with Applications

Ferragina and Manzini (2000). Proceedings of the 41st Annual Symposium on Foundations of Computer Science. p.390.

Exact Matching Review & Overview

Where is GATTACA in the human genome?

Brute Force (3 GB)

BANANA
BAN
ANA
NAN
ANA

$O(m * n)$

Slow & Easy

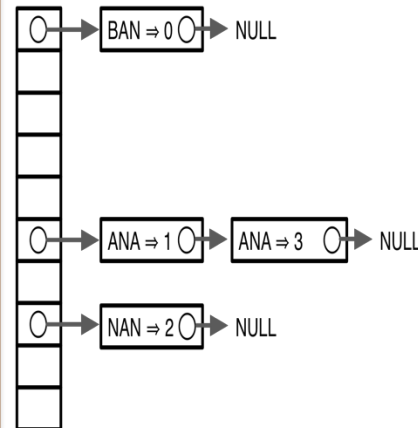
Suffix Array (>15 GB)

6	\$
5	A\$
3	ANA\$
1	ANANA\$
0	BANANA\$
4	NA\$
2	NANA\$

$O(m + \lg n)$

Full-text index

Hash Table (>15 GB)



$O(1)$

Fixed-length lookup

BWT (3 GB)

BANANA\$
=>
\$BANANA**A**
A\$BANAN**N**
ANASBAN**N**
ANANAS**B**
BANANA**\$**
NASBAN**A**
NANASB**A**
=>
ANNB\$AA

$O(m)$

Full-text and concise

*** These are general techniques applicable to any text search problem ***

Algorithm Overview

1. Split read into segments

Read

CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA

Read (reverse complement)

TACAGGCCTGGGTAAAATAAGGCTGAGAGCTACTGG

Policy: extract 16 nt seed every 10 nt

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: TCAGCCTTATTTACC

+, 20: **TTACCCAGGCCTGTA**

-, 0: **TACAGGCCTGGGTAAA**

-, 10: GGTAAAATAAGGCTGA

-, 20: GGCTGAGAGCTACTGG

2. Lookup each segment and prioritize

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: **TCAGCCTTATTTACC**

+, 20: **TTTACCCAGGCTGTA**

-, 0: **TACAGGCCTGGGTAAA**

-, 10: GGTAAAATAAGGCTGA

-, 20: GGCTGAGAGCTACTGG

Ungapped
alignment with
FM Index

Diagram illustrating a sequence alignment between a query sequence (I c g) and a reference sequence (a a c). The query sequence is shown in blue, and the reference sequence is shown in red. Red arrows indicate the alignment of 'c' in the query to 'c' in the reference, and 'g' in the query to 'a' in the reference.

Seed alignments (as B ranges)

$$\{ [211, 212], [212, 214] \}$$
$$\{ [653, 654], [651, 653] \}$$

{ [684, 685] }

{ }

{ }

{ [624, 625] }

3. Evaluate end-to-end match

Extension candidates

SA:684, chr12:1955

SA:624, chr2:462

SA:211: chr4:762

SA:213: chr12:1935

SA:652: chr12:1945

SIMD dynamic
programming
aligner

SAM alignments

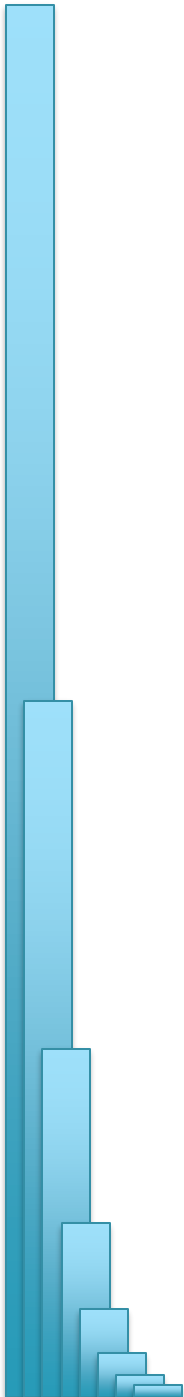
```

r1      0      chr12      1936      0
      36M      *      0      0
      CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA
      IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
      AS:i:0      XS:i:-2      XN:i:0
      XM:i:0      XO:i:0      XG:i:0
      NM:i:0      MD:Z:36      YT:Z:UU
      YM:i:0

```

□ □

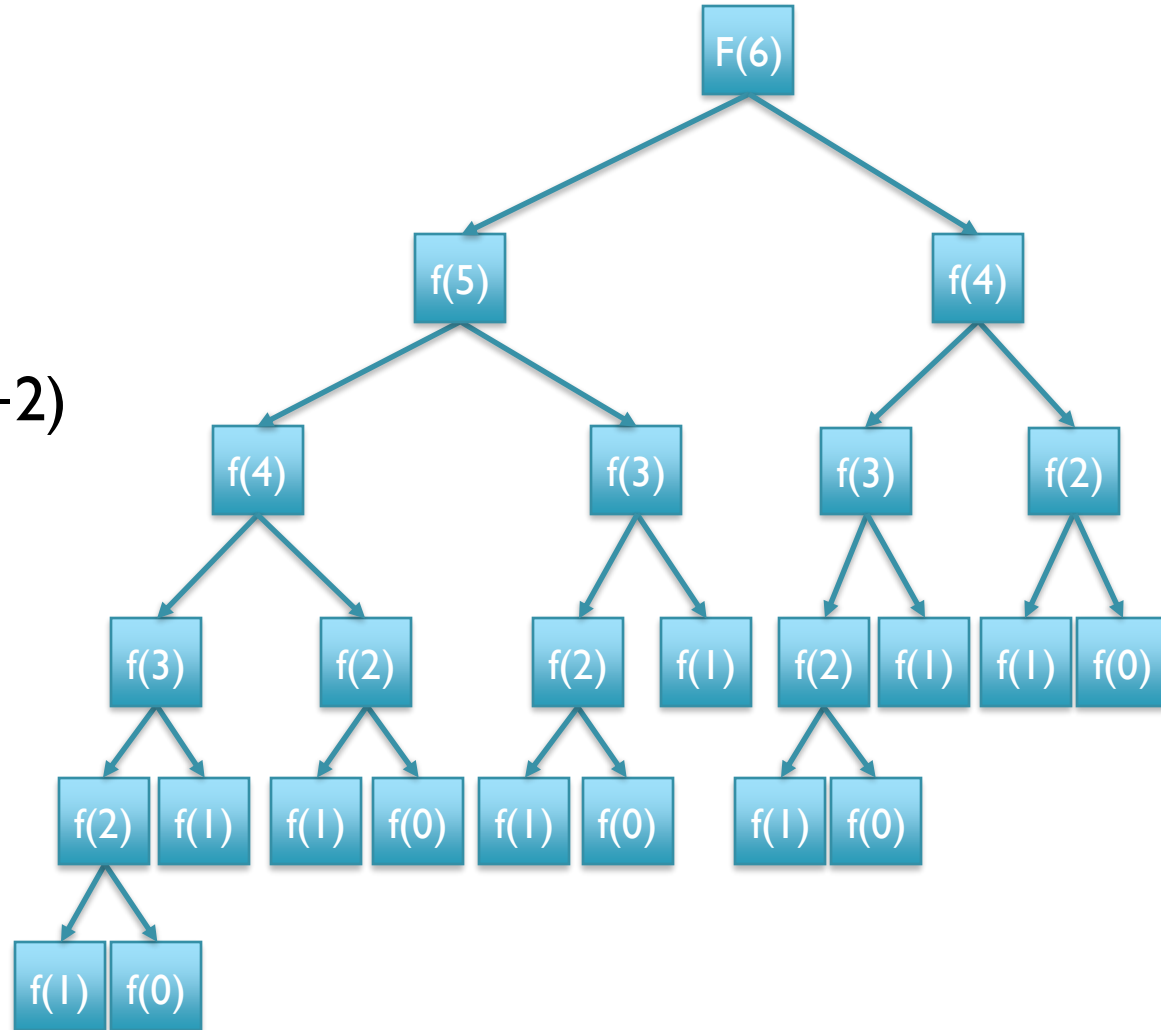
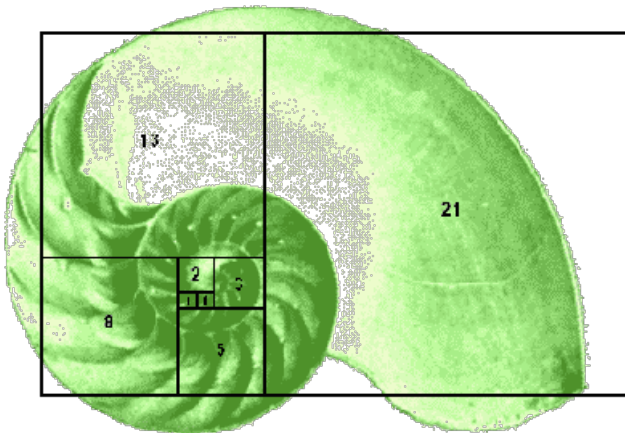
(Langmead & Salzberg, 2012)



Dynamic Programming

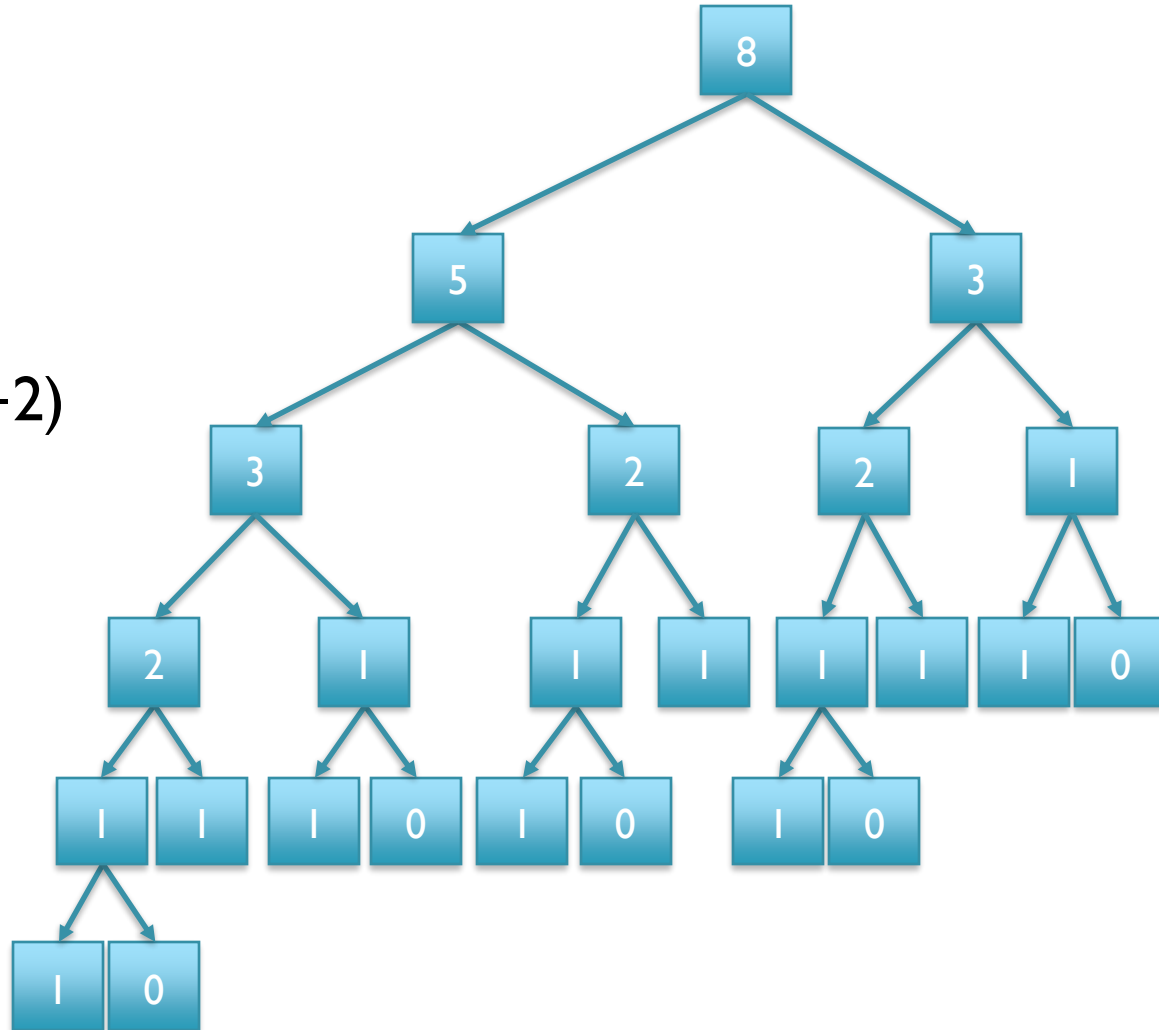
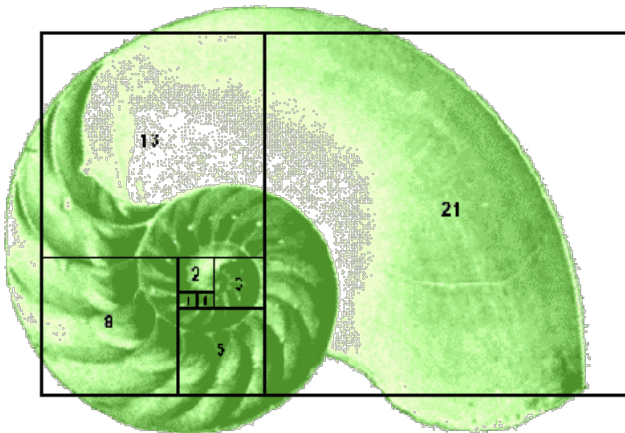
Fibonacci Sequence

```
def fib(n):  
    if n == 0 or n == 1:  
        return n  
    else:  
        return fib(n-1) + fib(n-2)
```



Fibonacci Sequence

```
def fib(n):  
    if n == 0 or n == 1:  
        return n  
    else:  
        return fib(n-1) + fib(n-2)
```



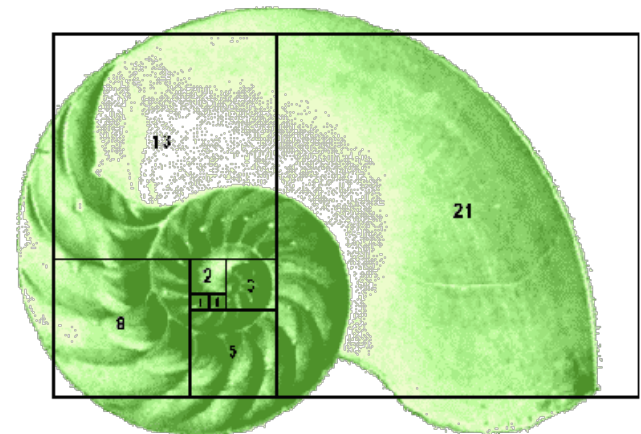
[What is the running time?]

Bottom-up Fibonacci Sequence

```
def fib(n):  
    table = [0] * (n+1)  
    table[0] = 0  
    table[1] = 1  
    for i in range(2,n+1):  
        table[i] = table[i-2] + table[i-1]  
    return table[n]
```

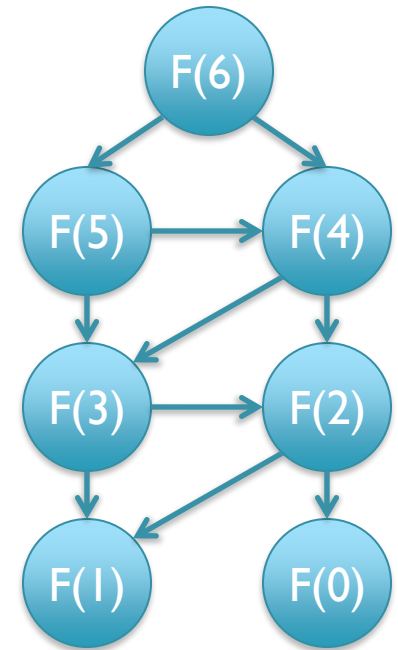
0	1	2	3	4	5	6
0	1	1	2	3	5	8

[What is the running time?]



Dynamic Programming

- General approach for solving (some) complex problems
 - When applicable, the method takes far less time than naive methods.
 - Polynomial time ($O(n)$ or $O(n^2)$) instead of exponential time ($O(2^n)$ or $O(3^n)$)
- Requirements:
 - **Overlapping subproblems**
 - **Optimal substructure**
- Applications:
 - Fibonacci
 - Longest Increasing Subsequence (Bonus Slides!)
 - Sequence alignment, Dynamic Time Warp, Viterbi
- Not applicable:
 - Traveling salesman problem, Clique finding, Subgraph isomorphism, ...
 - The cheapest flight from airport A to airport B involves a single connection through airport C, but the cheapest flight from airport A to airport C involves a connection through some other airport D.



Similarity metrics

- Hamming distance

- Count the number of substitutions to transform one string into another

MIKESCHATZ

| | X | | XXXX |

MICESHATZZ

5

- Edit distance

- The minimum number of substitutions, insertions, or deletions to transform one string into another

MIKESCHAT-Z

| | X | | X | | | X |

MICES-HATZZ

3

Edit Distance Example

AGCACACA → ACACACTA in 4 steps

AGCACACA → (1. change G to C)

ACCACACA → (2. delete C)

ACACACA → (3. change A to T)

ACACACT → (4. insert A after T)

ACACACTA → done

[Is this the best we can do?]

Edit Distance Example

AGCACACA → ACACACTA in 3 steps

AGCACACA → (1. change G to C)

ACCACACA → (2. delete C)

ACACACA → (3. insert T after 3rd C)

ACACACTA → done

[Is this the best we can do?]

Reverse Engineering Edit Distance

$$D(\text{AGCACACA}, \text{ACACACTA}) = ?$$

Imagine we already have the optimal alignment of the strings, the last column can only be 1 of 3 options:

... M	... I	... D
...A	...-	...A
...A	...A	...-

The optimal alignment of last two columns is then 1 of 9 possibilities

.. MM	.. IM	.. DM	.. MI	.. II	.. DI	.. MD	.. ID	.. DD
..CA	..-A	..CA	..A-	..--	..A-	..CA	..-A	..CA
..TA	..TA	..-A	..TA	..TA	..-A	..A-	..A-	..--

The optimal alignment of the last three columns is then 1 of 27 possibilities...

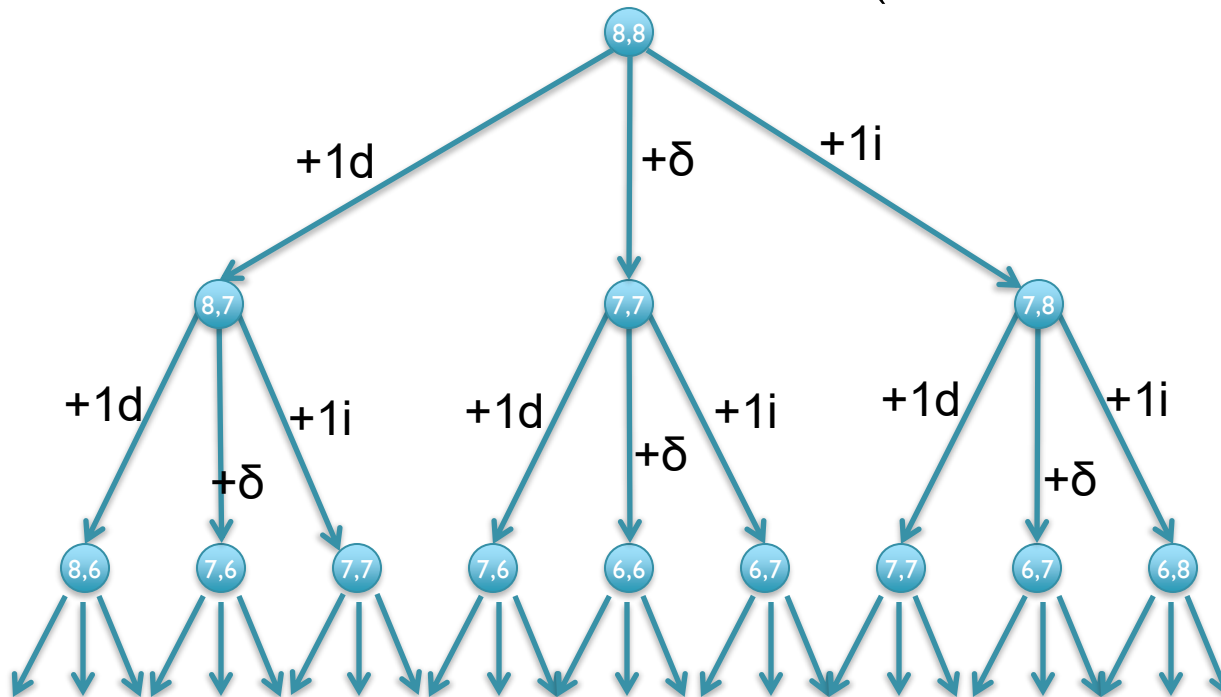
.. M ...	.. I ...	.. D ...
...X...	...-...	...X...
...Y...	...Y...	...-...

Eventually spell out every possible sequence of {I,M,D}

Recursive solution

- Computation of D is a recursive process.
 - At each step, we only allow matches, substitutions, and indels
 - $D(i,j)$ in terms of $D(i',j')$ for $i' \leq i$ and $j' \leq j$.

$$\begin{aligned} D(\text{AGCACACA}, \text{ACACACTA}) = \min \{ & D(\text{AGCACACA}, \text{ACACACT}) + 1, \\ & D(\text{AGCACAC}, \text{ACACACTA}) + 1, \\ & D(\text{AGCACAC}, \text{ACACACT}) + \delta(\text{A}, \text{A}) \} \end{aligned}$$



[What is the running time?]